

Documentation de DuckyRoad

Application de navigation web



Duckyroad

Table des matières

1	Analyse préliminaire	3
1.1	Introduction	3
1.2	Glossaire	3
1.3	Objectifs.....	4
1.4	Planification initiale	9
2	Analyse / Conception.....	12
2.1	Concept	12
2.2	Stratégie de test.....	14
2.3	Risques techniques	15
2.4	Planification	16
2.5	Dossier de conception	16
2.5.1	Dictionnaire de données	24
2.6	Justification des choix technologiques.....	27
3	Réalisation.....	28
3.1	Dossier de réalisation	28
3.1.1	Architecture et Répertoires d'installation.....	28
3.1.2	Liste des fichiers importants et leur rôle	29
3.1.3	Environnement logiciel Backend.....	30
3.1.4	Sécurité et logique de l'API REST	30
3.1.5	Version du projet.....	31
3.1.6	Interfaces Utilisateur	31
3.2	Description des tests effectués	34
3.3	Erreurs restantes	39
3.4	Liste des documents fournis	40
4	Conclusions	40
4.1	Objectifs atteints / non-atteints	40
4.2	Points positifs / négatifs	41
4.3	Difficultés particulières	41
4.4	Suites possibles pour le projet (évolutions & améliorations).....	42
5	Annexes.....	43
5.1	Résumé du rapport du TPI / version succincte de la documentation	43
5.2	Sources – Bibliographie.....	43
5.3	Journal de travail	44
5.4	Manuel d'Installation	47
5.5	Manuel d'Utilisation.....	48
5.6	Archives du projet.....	49
5.7	Liens vers les sites du projet :	49
5.8	Table des illustrations	49

1 Analyse préliminaire

1.1 Introduction

Dans le cadre de ma formation en informatique d'application au CPNV de Sainte-Croix, je dois réaliser un Travail Pratique Individuel (TPI). Ce projet de fin d'apprentissage me permet de mettre en pratique et de concrétiser l'ensemble des compétences acquises durant mon cursus, tant au niveau du développement frontend et backend que de la gestion de base de données.

Le projet consiste à développer "DuckyRoad", une application web de navigation et de signalement collaboratif en temps réel, fortement inspirée de Waze. L'objectif principal n'est pas de reproduire un système de navigation complet et complexe, mais de concevoir un *Minimum Viable Product* (MVP) fonctionnel. Ce MVP doit démontrer la maîtrise de concepts clés tels que la cartographie, la géolocalisation, le calcul d'itinéraires et la collaboration en temps réel entre les utilisateurs.

L'application est conçue avec une architecture full-stack, comprenant un front-end web optimisé pour les supports mobiles (HTML5, CSS, JavaScript), un backend API (Node.js/Express), et une base de données relationnelle MySQL. De plus, l'application exploite une technique de requêtage régulier (Polling) pour assurer une synchronisation dynamique et interactive des incidents routiers entre tous les utilisateurs connectés, permettant de simuler efficacement le temps réel.

1.2 Glossaire

Afin de faciliter la lecture de ce document pour des personnes non initiées à certaines technologies spécifiques au développement web et à la cartographie, voici les définitions des termes techniques employés dans ce rapport :

- **API REST (Representational State Transfer)** : Interface de programmation permettant au frontend (client) de communiquer avec le backend (serveur) de manière standardisée via des requêtes HTTP (GET, POST, etc.).
- **Clustering** : Algorithme de regroupement visuel. Sur une carte, il permet de fusionner plusieurs points très proches géographiquement en une seule icône afin d'éviter la surcharge visuelle.
- **Debounce** : Technique de programmation permettant de limiter la fréquence à laquelle une fonction est appelée. Utilisée ici pour ne pas inonder l'API de requêtes lors de la saisie rapide d'une adresse.
- **Endpoint** : Point de terminaison (une URL spécifique) d'une API sur lequel un client peut se connecter pour envoyer ou récupérer des données (ex: /api/login).

- **Glassmorphism** : Tendance de design d'interface utilisateur (UI) caractérisée par des effets de transparence et de flou d'arrière-plan, donnant l'apparence de verre givré.
- **JWT (JSON Web Token)** : Standard ouvert permettant d'échanger des informations de manière sécurisée sous forme de jeton crypté. Il est utilisé dans ce projet pour maintenir l'utilisateur connecté de manière sécurisée.
- **Leaflet** : Bibliothèque JavaScript open-source reconnue pour sa légèreté et son efficacité, utilisée pour l'affichage de cartes interactives.
- **MVP (Minimum Viable Product)** : Version d'un produit ne comportant que les fonctionnalités essentielles, permettant de valider rapidement le concept auprès des utilisateurs avec un minimum d'efforts de développement.
- **Polling** : Technique de communication où le client (navigateur) interroge le serveur à intervalles réguliers (ex: toutes les 15 secondes) pour vérifier s'il y a de nouvelles données (comme de nouveaux incidents).
- **WebSockets** : Protocole réseau permettant de créer un canal de communication bidirectionnel et en temps réel entre le navigateur et le serveur.

1.3 Objectifs

Les objectifs de ce projet sont définis par le cahier des charges et se divisent en plusieurs fonctionnalités clés qui devront être validées à la fin du développement.

Voici le cahier des charges :

1 INFORMATIONS GENERALES

Candidat :	Nom : DELAFontaine	Prénom : NIELS
	✉ : niels.delafontaine@eduvaud.ch	☎ : 079 312 77 31
Lieu de travail :	☐ CPNV, Rue de la Gare 14, 1450 Sainte-Croix	
Orientation :	☐ 88601 Développement d'application ☐ 88602 Informatique d'entreprise ☐ 88603 Technique des systèmes	
Chef de projet :	Nom : SAISON	Prénom : Yann
	✉ : yann.saison@eduvaud.ch	☎ : 079 610 08 47

2 PROCÉDURE

Le candidat réalise un travail personnel sur la base d'un cahier des charges reçu le 1er jour.

Le cahier des charges est approuvé par les deux experts. Il est en outre présenté, commenté et discuté avec le candidat. Par sa signature, le candidat accepte le travail proposé.

Le candidat a connaissance de la feuille d'appréciation avant de débiter le travail.

Le candidat est entièrement responsable de la sécurité de ses données.

En cas de problèmes graves, le candidat avertit au plus vite les deux experts et son CdP.

Le candidat a la possibilité d'obtenir de l'aide, mais doit le mentionner dans son dossier.

A la fin du délai imparti pour la réalisation du TPI, le candidat doit transmettre par courrier électronique le dossier de projet aux deux experts et au chef de projet. En parallèle, une copie papier du rapport doit être fournie sans délai en trois exemplaires (L'un des deux experts peut demander à ne recevoir que la version électronique du dossier). Cette dernière doit être en tout point identique à la version électronique.

Figure 1 Cahier des charges

3 TITRE

Développement d'une application web de navigation et de signalement collaboratif en temps réel.

4 MATÉRIEL ET LOGICIEL À DISPOSITION

- 1 ordinateur type CPNV
- Environnement de développement Web
- Outil de modélisation de base de données
- Logiciels de la suite Microsoft Office pour la rédaction du rapport et la présentation

5 PRÉREQUIS

- Développement Web (HTML5, CSS, PHP, JavaScript)
- Modélisation et gestion de base de données (MySQL)

6 DESCRIPTIF DU PROJET

Ce projet consiste à développer une application web inspirée de Waze, permettant aux utilisateurs d'afficher leur position sur une carte, d'obtenir un itinéraire simple et de signaler des incidents présents sur la route (bouchons, travaux, accidents, dangers). L'objectif n'est pas de reproduire un système complexe de navigation complet, mais de réaliser un MVP (*Minimum Viable Product*) fonctionnel démontrant les concepts clés : cartographie, localisation, routage, collaboration en temps réel et interaction utilisateur.

L'application sera accessible depuis un navigateur, optimisée pour mobile, et reposera sur une architecture full-stack comprenant un front-end web, un backend API, et une base de données. Les mises à jour d'incidents seront synchronisées entre utilisateurs grâce à des WebSockets, afin de fournir une expérience dynamique et interactive.

Principales fonctionnalités :

1. Affichage de la carte via un service de tuiles (OpenStreetMap, Mapbox ou équivalent).
2. Localisation de l'utilisateur grâce à l'API de géolocalisation du navigateur.
3. Recherche et calcul d'itinéraire simple (point A → point B) via une API de routage.
4. Signalement d'incidents (accident, embouteillage, travaux, véhicule arrêté, etc.).
5. Visualisation des incidents sur la carte sous forme d'icônes.
6. Mises à jour en temps réel des positions ou signalements via WebSockets (Socket.io).
7. Interface web responsive utilisable depuis smartphone.
8. Stockage en base de données des incidents, avec historique limité dans le temps.

Figure 2 Cahier des charges

7 LIVRABLES

Le candidat est responsable de livrer à son chef de projet :

Au démarrage du projet :

- Une planification initiale au format PDF.
- Le modèle de données.

Durant le temps alloué au projet, 1x par semaine :

- Un rapport de projet au format PDF.
- Un journal de travail au format PDF.
- Un livrable de l'application web déposé idéalement sur un dépôt distant de type Github ou Bitbucket.

A la fin du projet :

- Le rapport de projet final et son journal de travail imprimés.
- Le rapport de projet final et son journal de travail au format PDF.
- Un livrable de l'application web déposé idéalement sur un dépôt distant de type Github ou Bitbucket.
- Un fichier « archive » contenant :
 - Un script de création de la base de données SQL.
 - Un dossier contenant l'application complète.
 - Une procédure d'installation et de mise en service de l'application.

8 POINTS TECHNIQUES ÉVALUÉS SPÉCIFIQUES AU PROJET

La grille d'évaluation définit les critères généraux selon lesquels le travail du candidat sera évalué (documentation, journal de travail, respect des normes, qualité, ...).

En plus de cela, le travail sera évalué sur les 7 points spécifiques suivants (Point A14 à A20) :

- 1 La modélisation de l'application.
- 2 Le fonctionnement de l'infrastructure mise en place (backend / frontend)
- 3 La pertinence et la justification des choix technologiques effectués.
- 4 L'affichage de la carte et la géolocalisation.
- 5 Le calcul d'itinéraires.
- 6 Le signalement des incidents.
- 7 La description et la qualité des tests effectués.

9 VALIDATION

	Lu et approuvé le :	Signature :
Candidat :	10.02.2026	N. Delafontaine
Chef de projet :		

Figure 3 Cahier des charges

Fonctionnalités générales de l'application :

- Interface web responsive, pensée et optimisée pour une utilisation depuis un smartphone.
- Synchronisation et mise à jour quasi-instantanée des positions et des signalements grâce à l'utilisation d'un système de requêtage régulier (Polling), garantissant une expérience collaborative réactive.
- Stockage persistant des incidents dans la base de données, avec un système d'historique limité dans le temps pour garder une carte pertinente.
- Dictionnaire de langue pour pouvoir changer la langue.

Fonctionnalités cartographiques et de navigation :

- Affichage fluide d'une carte interactive via un service de tuiles tel qu'OpenStreetMap.
- Localisation précise de l'utilisateur en exploitant l'API de géolocalisation native du navigateur.
- Possibilité d'effectuer une recherche et de calculer un itinéraire simple (d'un point A à un point B) en s'appuyant sur une API de routage.

Signalement collaboratif :

- Système permettant à l'utilisateur de signaler divers incidents rencontrés sur la route (accidents, embouteillages, travaux, véhicule arrêté, problèmes de visibilité, police, etc.).
- Visualisation claire de ces incidents directement sur la carte par l'intermédiaire d'icônes personnalisées.

Points techniques spécifiques évalués : En plus des critères généraux, le projet sera particulièrement évalué sur les points suivants :

- Le fonctionnement global de l'infrastructure mise en place (lien entre backend et frontend).
- La pertinence et la justification des choix technologiques effectués.
- L'affichage de la carte couplé à la géolocalisation.
- La modélisation de l'application et de sa base de données.
- L'implémentation du calcul d'itinéraires.
- Le système de signalement des incidents et sa réactivité.

- La description ainsi que la qualité des tests effectués pour valider l'application.

1.4 Planification initiale

Pour organiser mon travail et suivre mon avancement, j'ai choisi d'utiliser l'outil de gestion de projet Agile **IceScrum**.

L'intégralité du développement a été configurée sous la forme d'un grand sprint (Sprint 1), regroupant l'ensemble des tâches (*User Stories*) nécessaires à l'aboutissement du projet (Proof of Concept / MVP).

Voici la liste détaillée des tâches planifiées, extraites de mon tableau IceScrum, organisées par catégories logiques de développement :

Catégorie	Tâche planifiée (User Story)	Nombres d'heures estimées	Nombres d'heures réalisées
Infrastructure & BDD	Création de la base de données (Create DB)	1	3
	Connexion à la base de données (Link with DB)	1	1.5
Authentification	Interface de connexion et d'inscription (UI for login and register)	2	3
	Inscription d'un utilisateur (Register)	2	1
	Connexion d'un utilisateur (Login)	2	1
	Gestion des jetons d'authentification (Gestion of token)	1	1.5
	Suppression du compte (Deletion of account)	1	0.5

Catégorie	Tâche planifiée (User Story)	Nombres d'heures estimées	Nombres d'heures réalisées
Cartographie & UI	Intégration de la carte (Get map)	2	1.5
	Affichage et actualisation de l'utilisateur (User is shown and actualisation)	4	2.5
	Rotation de la carte (Rotation of map)	2	2.5
	Maintien de l'écran allumé (Screen stay awake)	2	1
	Amélioration globale de l'interface (Improve UI)	2	1
	Gestion du multilinguisme (Gestion of language)	3	4.5
	Affichage des données diverses (Print datas infos)	2	1
Navigation GPS	Récupération de la position de l'utilisateur (Get user position)	2	1.5
	Géocodage d'une adresse (Geocoding an address)	2	3
	Suggestion / Autocomplétion d'adresses (Suggestion of location)	3	2

Catégorie	Tâche planifiée (User Story)	Nombres d'heures estimées	Nombres d'heures réalisées
	Création de l'itinéraire de navigation (Create navigation road)	3	3.5
	Démarrage / Activation de la navigation (Activation of navigation)	2	2
	Affichage de la vitesse actuelle (Get current speed)	2	3
Fonctionnalités métier	Sauvegarder un lieu favori (Save important place)	2	1.5
	Récupérer et afficher les lieux favoris (Get important place)	3	3
	Signaler un incident (Register incident)	2	2
	Récupérer et afficher les incidents sur la carte (Get incident)	2	2
Transverse	Rédaction de la documentation (Documentation)	16	19
	Tests fonctionnels (Test functions)	1	2
	Correction des bugs (Fix bugs)	3	5

Catégorie	Tâche planifiée (User Story)	Nombres d'heures estimées	Nombres d'heures réalisées
	Total :	70 heures	75 heures

2 Analyse / Conception

2.1 Concept

Le contexte et l'application Le projet consiste à développer une application web nommée "DuckyRoad", inspirée de Waze. Il s'agit d'un MVP (Minimum Viable Product) permettant aux utilisateurs de se géolocaliser, de calculer un itinéraire et de signaler des incidents en temps réel de manière collaborative. L'application est pensée "Mobile-First" pour être utilisée facilement depuis un smartphone en voiture.

Multimédia : Gabarits et flux utilisateur

L'interface utilisateur est scindée en plusieurs vues épurées, utilisant un design "Glassmorphism" pour une sensation moderne et intégrée par-dessus la cartographie. Le flux de navigation (storyboard) a été défini comme suit :

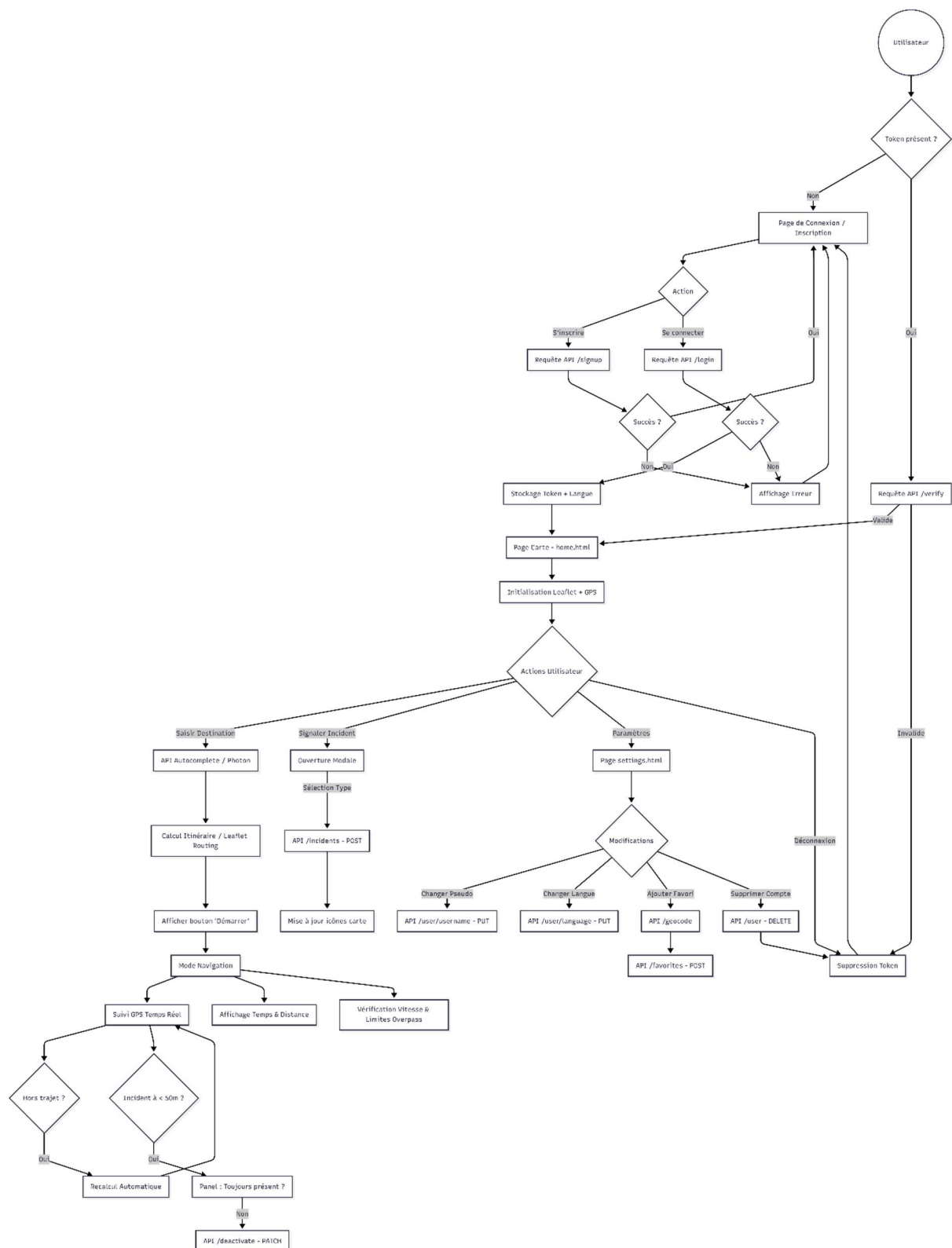


Figure 4 FlowChart

Un utilisateur arrive sur l'application et est invité à se connecter ou à créer un compte. Une fois authentifié, il atterrit sur la page d'accueil (Home Page) affichant la carte et sa position.

Il peut entrer une destination manuellement ou choisir un favori.
Pendant le trajet, il a la possibilité d'interagir avec la carte pour signaler des incidents (accidents, police, travaux, etc.) ou être alerté en cas d'excès de vitesse.

Bases de données : Modèle Conceptuel

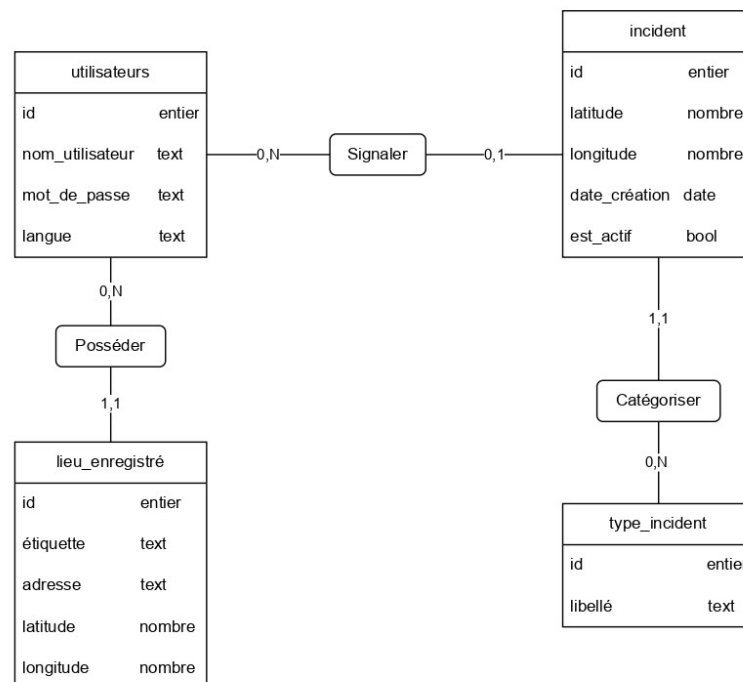


Figure 5 MCD

L'application repose sur une base de données relationnelle simple mais efficace, gérant les éléments suivants :

Utilisateurs : Gestion de l'authentification et des préférences (langue).

Incidents : Enregistrement des signalements avec leurs coordonnées GPS, leur type et leur statut d'activité (actif/inactif).

Favoris (Saved Locations) : Lieux sauvegardés par l'utilisateur pour une navigation rapide.

Programmation : Architecture L'application suit une architecture Full-Stack:

Frontend : Développé en HTML/CSS/JavaScript vanilla. Il intègre la librairie Leaflet.js pour l'affichage de la carte et Leaflet Routing Machine pour le calcul d'itinéraires.

Backend : Une API REST développée en Node.js avec Express.js. Elle est chargée de faire la liaison avec la base de données MySQL via le module mysql2.

Sécurité : L'authentification des requêtes se fait via des JSON Web Tokens (JWT) stockés côté client.

2.2 Stratégie de test

La stratégie de test mise en place a pour but de garantir la fiabilité de l'application, en particulier pour les fonctionnalités critiques comme le routage et le signalement d'incidents.

- **Types de tests et ordre d'exécution :**
 1. *Tests unitaires (Backend)* : Test manuel des routes de l'API (Inscription, Connexion, Ajout d'incident, Recherche d'adresse) à l'aide d'outils comme Postman pour vérifier le retour JSON et la bonne insertion en base de données.
 2. *Tests d'intégration (Frontend/Backend)* : Vérification de la communication entre l'interface utilisateur et l'API (ex: connexion réussie redirigeant vers la carte).
 3. *Tests fonctionnels (Terrain)* : Test de la géolocalisation en conditions réelles (en mouvement avec un smartphone) pour valider le suivi GPS, la rotation de la carte en fonction du cap, et les alertes de limitation de vitesse.
- **Moyens à mettre en œuvre** : Navigateurs web (Chrome, Firefox) avec outils de développement pour simuler différentes tailles d'écrans. Test en direct sur un appareil mobile physique connecté au serveur (ou à l'hébergement distant).
- **Couverture des tests** : Les tests ne seront pas exhaustifs en raison de la contrainte de temps (90h au total). L'accent sera mis sur les scénarios nominaux (le "Happy Path" de la connexion jusqu'à l'arrivée à destination) et la gestion des erreurs de l'API (ex: adresse introuvable, jeton expiré).
- **Données de test** : Des utilisateurs fictifs seront créés. Les incidents seront générés manuellement dans un rayon proche de la position de test (ex: Yverdon ou Sainte-Croix) pour valider le système de clustering (regroupement des icônes sur la carte) et les pop-ups de vérification ("L'incident est-il toujours présent ?").
- **Testeurs extérieurs** : Quelques collègues ou apprentis seront sollicités pour tester l'interface sur leur propre téléphone afin d'obtenir un retour sur l'ergonomie (UX/UI).

2.3 Risques techniques

Plusieurs risques techniques ont été identifiés avant et pendant le développement, nécessitant la mise en place de solutions spécifiques :

Dépendance aux API externes (Géocodage et Routage) : L'application utilise *Nominatim* (OpenStreetMap) et *Photon* pour la recherche d'adresse, qui imposent des limites de requêtes et peuvent être lents.

Solution appliquée : Mise en place d'un système de "fallback". Si Nominatim échoue, le backend interroge l'API Photon. De plus, un délai (setTimeout de 400ms) a été ajouté (Debounce) sur l'autocomplétion pour éviter de surcharger l'API externe à chaque touche pressée.

Précision de la géolocalisation et rotation : Gérer les caprices du GPS des navigateurs mobiles (perte de signal, données de cap erronées).

Solution appliquée : Utilisation de watchPosition avec enableHighAccuracy: true. Un algorithme a été implémenté pour vérifier si l'utilisateur s'éloigne de plus de 50 mètres du tracé (isOffRoute), déclenchant ainsi un recalcul automatique de l'itinéraire.

Gestion du temps réel et surcharge visuelle : L'affichage simultané de nombreux incidents peut rendre la carte illisible.

Solution appliquée : Le cahier des charges prévoyait initialement des WebSockets, mais pour limiter la complexité et garantir la stabilité du MVP, un système de "Polling" (requête toutes les 15 secondes) a été mis en place. Un algorithme de "Clustering" a également été développé côté client pour fusionner visuellement les incidents situés à moins de 50 mètres les uns des autres.

2.4 Planification

La planification initiale a été revue pour s'adapter à la réalité du développement et aux difficultés techniques rencontrées. Initialement estimé à 70 heures de travail, le projet a finalement requis un total de 75 heures de réalisation, soit un dépassement global et maîtrisé de 5 heures.

Cet écart s'explique par les ajustements suivants :

- **Ajustement des délais techniques** : La mise en place de base de la carte avec Leaflet et Leaflet-Routing-Machine a été relativement rapide. En revanche, le développement de l'algorithme de recalcul d'itinéraire en direct (isOffRoute) et la récupération dynamique des limitations de vitesse via *Overpass API* ont été techniquement plus complexes et ont consommé plus de temps que prévu.
- **Redistribution du temps et justification du dépassement** : Face à ces défis, le temps initialement prévu pour concevoir un design complexe des paramètres a été volontairement raccourci. Cette économie de temps a permis de réallouer des ressources à la fiabilisation du backend de l'API et à la sécurisation des requêtes (middleware JWT) aussi à la finalisation de la documentation qui m'a pris plus de temps que prévu, des éléments critiques qui ont nécessité environ 10 heures supplémentaires par rapport à l'estimation de base. Le temps gagné sur l'interface graphique n'ayant pas suffi à compenser totalement cet investissement imprévu sur la sécurité backend, le projet s'est conclu sur ce dépassement final de 5 heures.

2.5 Dossier de conception

Choix matériel et logiciel

- **Matériel** : Ordinateur du CPNV pour le développement ainsi qu'un ordinateur personnel, et un smartphone personnel pour valider le comportement de la carte et du GPS en condition réelle, une moto pour tester sur route.
- **Système d'exploitation** : Environnement de développement sous Windows. L'application cible les navigateurs iOS (Safari), Android (Chrome/Brave) et Edge.

- **Outils logiciels** : Environnement Node.js pour le serveur, IDE (WebStorm) pour le code, et HeidiSQL pour la modélisation et visualisation de la base de données MySQL.

Site Web et Architecture d'Hébergement

L'application est divisée en trois vues principales (index.html pour l'authentification, home.html pour la carte interactive, et settings.html pour les paramètres). L'architecture du projet est distribuée sur deux hébergeurs distincts :

- **Swisscenter** : Héberge les fichiers statiques du frontend (HTML/CSS/JS) ainsi que la base de données MySQL.
- **Render** : Héberge le backend (l'API Node.js/Express).

Le fonctionnement est le suivant : le navigateur client charge les pages HTML depuis Swisscenter, puis le code JavaScript effectue des requêtes HTTP (AJAX/Fetch) vers l'API hébergée sur Render, qui se charge à son tour de communiquer avec la base de données sur Swisscenter.

Bases de données : Modèle Logique (MLD)

Le schéma de la base de données est structuré autour des tables suivantes :

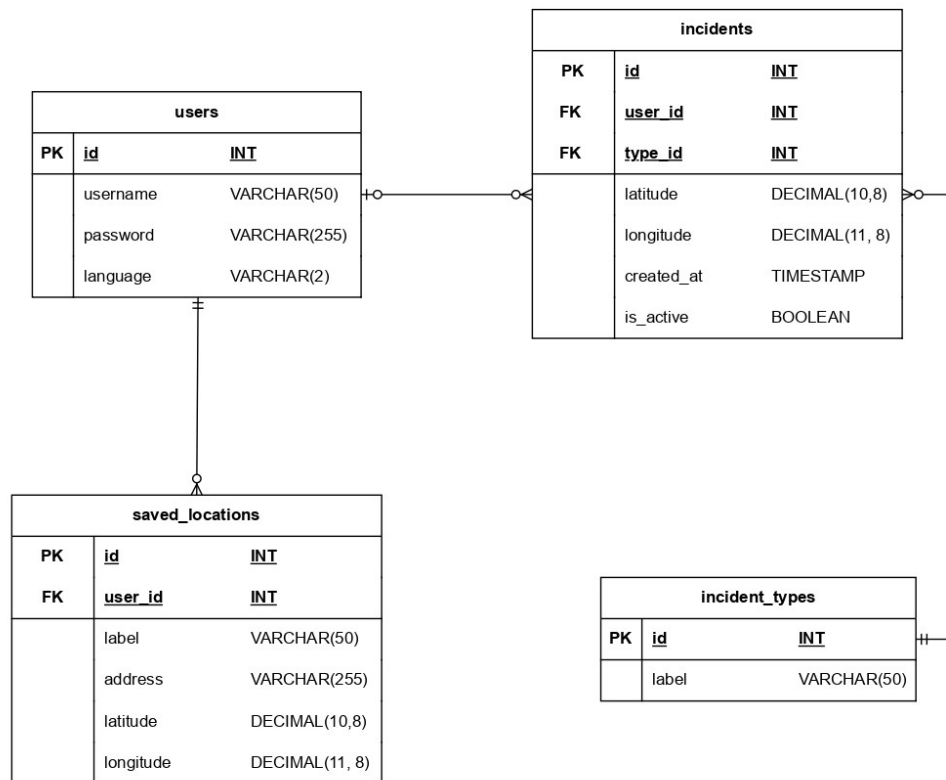


Figure 6 MLD

Maquettes de l'interface



Figure 7 Maquette Connexion



The image shows a mobile app registration screen for 'DuckyRoad'. At the top, a blue header bar contains the text 'DuckyRoad - Inscription'. Below this is a large, light gray rectangular area with a dashed border, labeled 'LOGO'. Underneath the logo area are two input fields: the first is labeled 'Nom d'utilisateur' and the second is labeled 'Mot de passe'. Below these fields is a prominent blue button with the text 'Inscription'. At the bottom of the screen, there is a link that reads 'Déjà un compte ? Connexion'. The entire interface is framed by a black border, and a gray horizontal bar at the very bottom represents the mobile device's home indicator.

Figure 8 Maquette Inscription

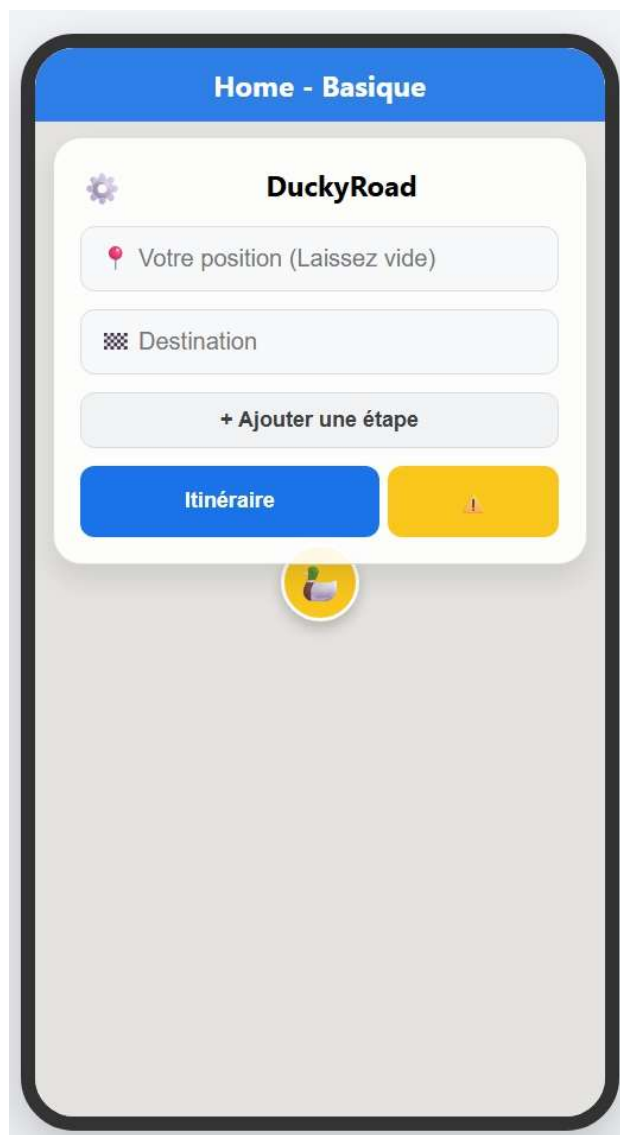


Figure 9 Maquette Home

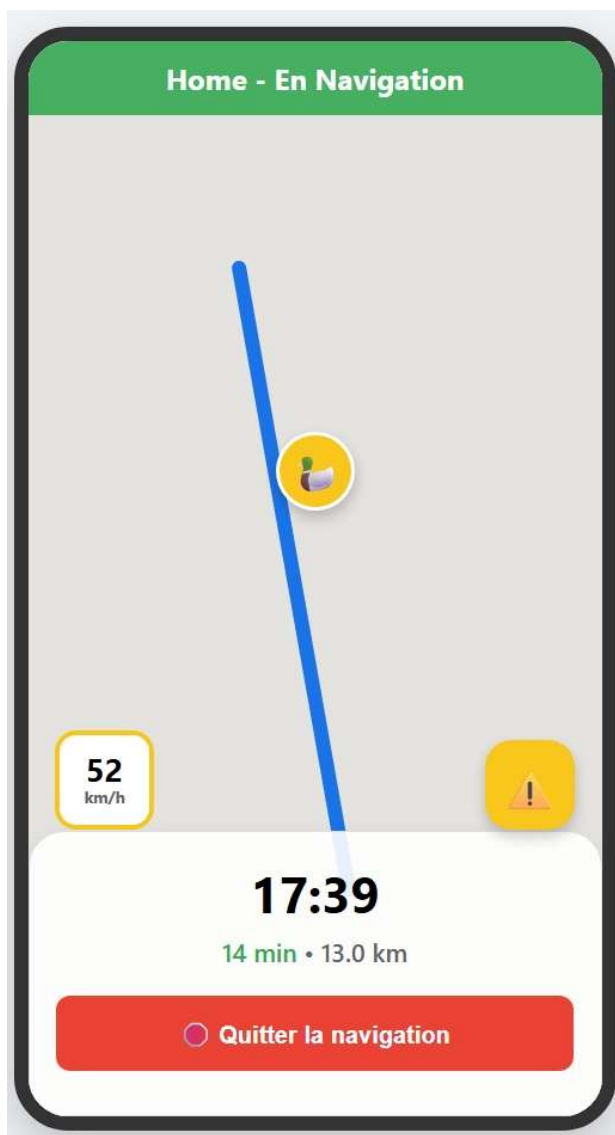


Figure 10 Maquette Navigation



Figure 11 Maquette Signalisation



Figure 12 Maquette Settings

2.5.1 Dictionnaire de données

Afin de détailler la structure de notre base de données relationnelle, voici le dictionnaire de données listant les tables, leurs champs respectifs, les types de données SQL choisis, ainsi que la justification de ces choix.

Table : users Cette table gère les comptes utilisateurs, leurs informations de connexion sécurisées et leurs préférences.

Nom du champ	Type SQL	Clé	Description et Raison du choix
id	INT	PK	Identifiant unique de l'utilisateur. Auto-incrémenté.
username	VARCHAR(50)		Nom d'utilisateur (pseudo) utilisé pour la connexion. Unique. Une limite de 50 caractères est largement suffisante.
password	VARCHAR(255)		Mot de passe de l'utilisateur. La taille de 255 caractères est requise pour stocker en toute sécurité la longue chaîne de caractères hachée générée par l'algorithme bcrypt.
language	VARCHAR(2)		Langue préférée de l'utilisateur pour l'interface (ex: 'fr', 'en', 'de'). Stocké sur 2 caractères en respectant la norme ISO 639-1.

Table : incident_types Cette table est un référentiel statique (catalogue) définissant les différentes catégories d'incidents possibles.

Nom du champ	Type SQL	Clé	Description et Raison du choix
id	INT	PK	Identifiant unique du type d'incident.
label	VARCHAR(50)		Nom explicite de l'incident (ex: 'Accident', 'Bouchon', 'Police'). 50 caractères conviennent parfaitement pour de courts libellés.

Table : incidents Cette table est le cœur du système collaboratif. Elle stocke tous les signalements remontés par les utilisateurs sur la carte.

Nom du champ	Type SQL	Clé	Description et Raison du choix
id	INT	PK	Identifiant unique de l'incident signalé.
user_id	INT	FK	Clé étrangère pointant vers users.id. Permet de tracer quel utilisateur a signalé l'incident.
type_id	INT	FK	Clé étrangère pointant vers incident_types.id. Relie le signalement à sa catégorie (icône à afficher).
latitude	DECIMAL(10,8)		Latitude GPS de l'incident. Le format DECIMAL(10,8) garantit une précision maximale sur la carte (précision au millimètre près).
longitude	DECIMAL(11,8)		Longitude GPS de l'incident (nécessite 11 chiffres au total pour gérer les coordonnées négatives larges).
created_at	TIMESTAMP		Date et heure exactes de la création du signalement. Nécessaire pour purger les anciens incidents ou gérer leur durée de vie.
is_active	BOOLEAN		Statut actuel de l'incident (Vrai/Faux). Permet de masquer de la carte les incidents vérifiés comme "résolus", sans pour autant les supprimer définitivement de l'historique de la base de données.

Table : saved_locations Cette table permet de stocker les adresses favorites personnalisées de chaque utilisateur pour accélérer la recherche d'itinéraire.

Nom du champ	Type SQL	Clé	Description et Raison du choix
id	INT	PK	Identifiant unique du lieu sauvegardé.
user_id	INT	FK	Clé étrangère pointant vers users.id. Relie directement un favori à son propriétaire.
label	VARCHAR(50)		Nom personnalisé donné par l'utilisateur (ex: 'Maison', 'Lieu de travail').
address	VARCHAR(255)		L'adresse textuelle complète pour l'affichage visuel dans les paramètres et les suggestions.
latitude	DECIMAL(10,8)		Coordonnée Y pré-géocodée du lieu favori. Évite de refaire un appel à l'API de géocodage externe à chaque fois.
longitude	DECIMAL(11,8)		Coordonnée X pré-géocodée du lieu favori.

Programmation et Scripts

- **Architecture Modulaire** : Le code backend est découpé logiquement. Le fichier principal app.js initialise le serveur Express et redirige les requêtes vers des fichiers de routage séparés (routes/auth.js pour les connexions, routes/navigation.js pour la carte et les incidents). Les requêtes SQL pures sont externalisées dans le dossier Database/ (LinkWithDatabase.js).
- **Sécurité** : Les mots de passe sont hachés avec le module bcrypt avant l'insertion en base de données.
- **Traduction (i18n)** : Côté frontend, un script de traduction a été implémenté (translations.js) permettant de basculer dynamiquement l'interface en Français, Anglais ou Allemand selon les paramètres de l'utilisateur, ce qui améliore considérablement l'accessibilité du MVP.

2.6 Justification des choix technologiques

Dans le cadre du développement de *DuckyRoad*, le choix des technologies a été soigneusement réfléchi afin de répondre aux contraintes d'un projet de type MVP, tout en garantissant des performances optimales et une facilité de maintenance :

- **Cartographie : Leaflet et OpenStreetMap (plutôt que Google Maps)** Le choix de s'appuyer sur la librairie *Leaflet* couplée aux tuiles gratuites d'*OpenStreetMap* s'est imposé pour plusieurs raisons. Contrairement à l'API de Google Maps qui nécessite l'enregistrement d'une carte de crédit et devient rapidement payante au-delà d'un certain quota, l'écosystème open-source d'OSM garantit une gratuité totale. De plus, Leaflet est extrêmement léger, pensé "Mobile-First", et offre une excellente flexibilité pour personnaliser l'interface (comme les icônes de "canards" pour le système de clustering).
- **Backend et API : Node.js avec le framework Express.js** L'utilisation de *Node.js* a permis de conserver une uniformité linguistique sur l'ensemble du projet : le JavaScript est utilisé à la fois côté client (Frontend) et côté serveur (Backend). Le framework *Express.js* a été sélectionné pour sa simplicité et sa robustesse lors de la création d'API REST. Son architecture orientée "middlewares" facilite grandement l'intégration de systèmes de sécurité (comme la validation des tokens JWT) et la gestion asynchrone des requêtes, un point crucial pour traiter de multiples interactions simultanées (géocodage, signalements).
- **Base de données : MySQL** Le choix de *MySQL* (base de données relationnelle) est justifié par la nature fortement liée des données de l'application. Un utilisateur possède des adresses favorites qui lui sont propres et crée des incidents. Ce modèle de données relationnel nécessite l'utilisation de clés étrangères pour garantir l'intégrité des informations. Bien qu'une base de données NoSQL aurait pu être envisagée, MySQL offre une structuration stricte et rassurante, particulièrement adaptée aux requêtes complexes nécessaires lors de l'authentification et de la récupération des historiques.

3 Réalisation

3.1 Dossier de réalisation

Ce chapitre détaille l'implémentation technique de l'application web de navigation, en se concentrant sur la logique côté serveur (backend) et l'API REST.

3.1.1 Architecture et Répertoires d'installation

Le backend de l'application a été développé avec Node.js et le framework Express. Il est conçu pour fonctionner comme une API RESTful qui sert les données au client et interagit avec la base de données. Pour la mise en production, l'architecture a été divisée de la manière suivante :

Serveur Web / API : L'application Node.js est hébergée et déployée sur la plateforme cloud Render.

Base de données : La base de données MySQL est hébergée sur les serveurs distants de Swisscenter.

Le code serveur se trouve à la racine du projet local PreTpi-Navigation-develop.

3.1.2 Liste des fichiers importants et leur rôle

Voici la description des fichiers clés développés pour l'infrastructure backend :

```
PreTpi-Navigation/
├── app.js           # Point d'entrée principal du serveur Node.js/Express
├── package.json     # Configuration du projet et dépendances (NPM)
├── .env             # Variables d'environnement (Non versionné : mots de passe,
clés secrètes)
├── Database/        # Gestion de la connexion et requêtes MySQL
│   ├── ConnectToDatabase.js
│   └── LinkWithDatabase.js
├── routes/          # Contrôleurs de l'API REST (Backend)
│   ├── auth.js      # Logique d'authentification (Login, Signup, Profil)
│   └── navigation.js # Logique cartographique (Incidents, Géocodage, Favoris)
├── views/           # Interface utilisateur (Frontend)
│   ├── index.html   # Vue de connexion / inscription
│   ├── home.html    # Vue principale de la carte et navigation
│   └── settings.html # Vue des paramètres utilisateurs
├── public/          # Scripts et ressources statiques du client
│   └── js/
│       └── translations.js # Dictionnaire de traduction (Fr, En, De)
├── Docs/            # Documentation et ressources visuelles
│   └── Images/
│       └── Edited/    # Icônes des incidents servies par l'API (canards)
```

Description détaillée des fichiers clés :

- **app.js** : C'est le cœur et le point d'entrée du serveur Node.js. Il a pour rôle d'initialiser l'application Express sur le port 3000. Il configure les middlewares vitaux comme les stratégies CORS (restreintes au domaine front-end tpi26nde.mycpnv.ch), expose les dossiers statiques, et orchestre le routage en redirigeant le trafic vers les modules appropriés (/api/ et /api/navigation/).
- **package.json** : Fichier fondamental de l'écosystème Node.js. Il définit les métadonnées du projet, les scripts de démarrage (ex: nodemon app.js pour le développement), ainsi que la liste stricte des dépendances requises (Express, Bcrypt, JsonWebToken, MySQL2).
- **routes/auth.js** : Contrôleur gérant l'intégralité du cycle de vie des utilisateurs. Ce fichier contient :
 - La logique d'inscription (/signup) et de connexion (/login).

- Le hachage cryptographique des mots de passe (via Bcrypt avec un *salt* de 10).
- La génération des tokens JWT pour l'authentification.
- Les routes de modification de profil (changement de pseudo, de langue) et de suppression de compte.
- **routes/navigation.js** : Contrôleur principal répondant aux exigences cartographiques du cahier des charges. Il a pour rôle de :
 - Recevoir et diffuser les signalements d'incidents (accidents, bouchons, etc.) et gérer leur désactivation.
 - Gérer les opérations CRUD pour les adresses favorites des utilisateurs.
 - Assurer le rôle de proxy pour le géocodage et l'autocomplétion en interrogeant dynamiquement les API externes (Nominatim/OpenStreetMap et Photon/Komoot), tout en fusionnant ces résultats avec les favoris locaux de l'utilisateur.
- **Database/LinkWithDatabase.js** : Interface faisant le pont entre les routes API et la base de données, contenant les requêtes SQL asynchrones préparées pour éviter les injections.
- **Database/ConnectToDatabase.js** : Script configurant le pool de connexion à la base de données MySQL distante (Swisscenter), utilisant les variables d'environnement (.env) pour sécuriser les accès.

3.1.3 Environnement logiciel Backend

Composant / Librairie	Technologie / Version	Rôle dans l'API REST
Environnement	Node.js (v20+)	Exécution du serveur backend.
Framework	Express.js	Création de l'API REST et gestion des routes HTTP.
Sécurité	JsonWebToken (JWT)	Génération et vérification des jetons d'accès.
Sécurité	Bcrypt	Hachage et salage des mots de passe en base de données.
Base de données	MySQL2 (npm)	Connecteur asynchrone (Promises) pour communiquer avec Swisscenter.

3.1.4 Sécurité et logique de l'API REST

L'API REST est conçue de manière *stateless* (sans état de session côté serveur). Lorsqu'un utilisateur se connecte via l'endpoint de login, l'API génère un JSON Web Token (JWT) signé avec une clé secrète stockée dans le fichier .env du serveur Render.

Ce token est renvoyé au client, qui doit ensuite l'inclure dans l'en-tête HTTP (Authorization: Bearer <token>) de chaque requête dirigée vers des endpoints protégés (comme la création d'un incident ou l'ajout d'un favori). Un middleware

backend intercepte ces requêtes, décode le JWT, vérifie sa validité et extrait l'ID de l'utilisateur pour autoriser l'action dans la base de données.

3.1.5 Version du projet

L'API REST et le backend associés sont actuellement déployés en version 1.0.0, les fonctionnalités de requêtage, de sécurité JWT et d'interaction avec la base de données distante étant totalement opérationnelles sur Render.

3.1.6 Interfaces Utilisateur

Voilà les images de l'interface utilisateur :



Figure 14 Inscription



Figure 13 Login

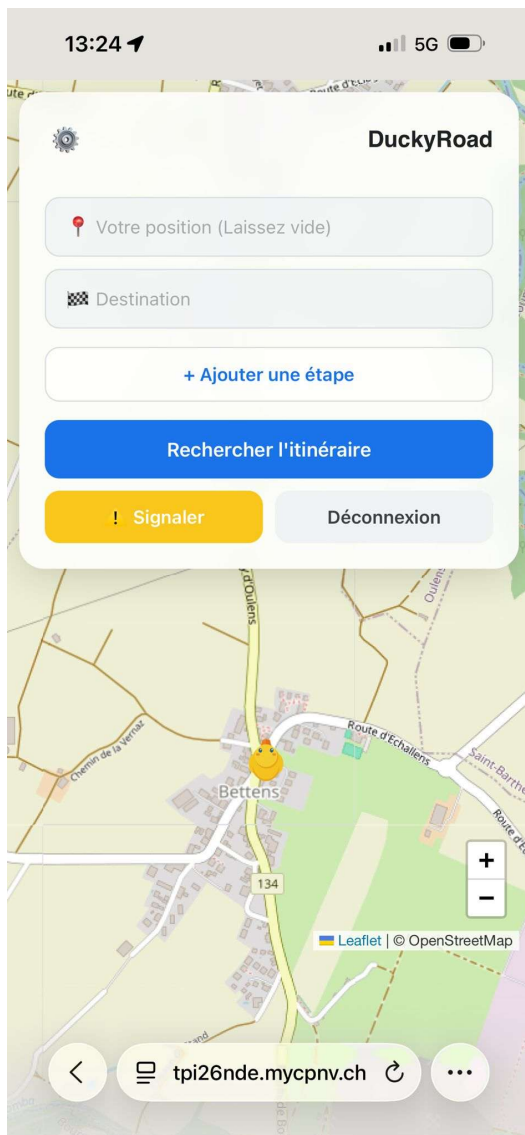


Figure 16 Home

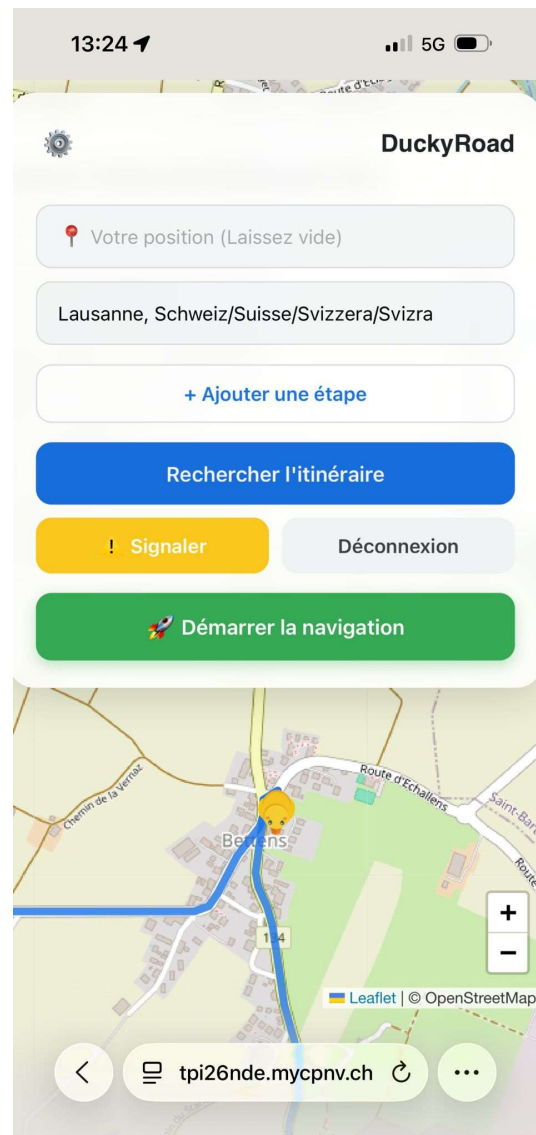


Figure 15 Start Navigation

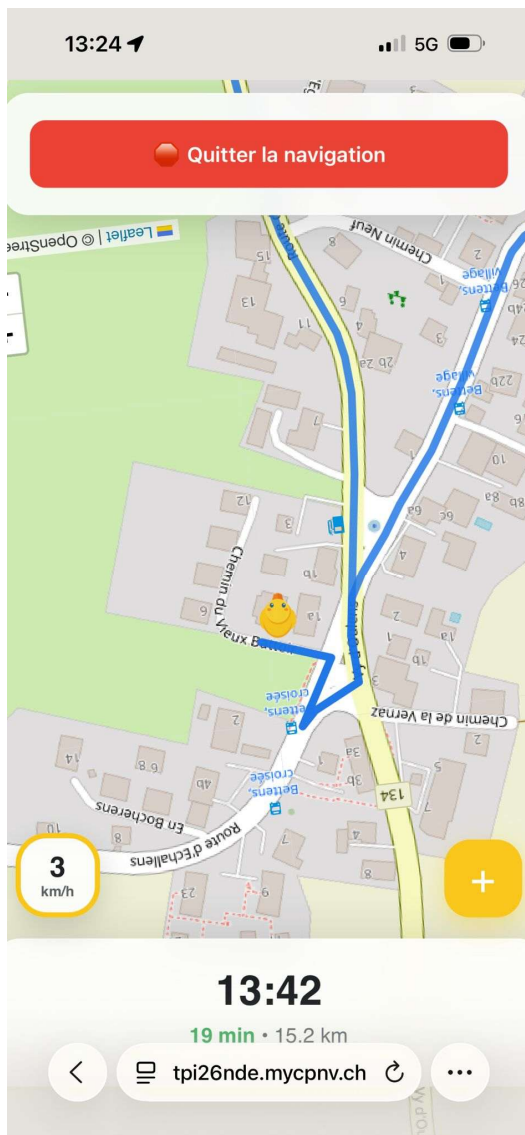


Figure 18 Navigation

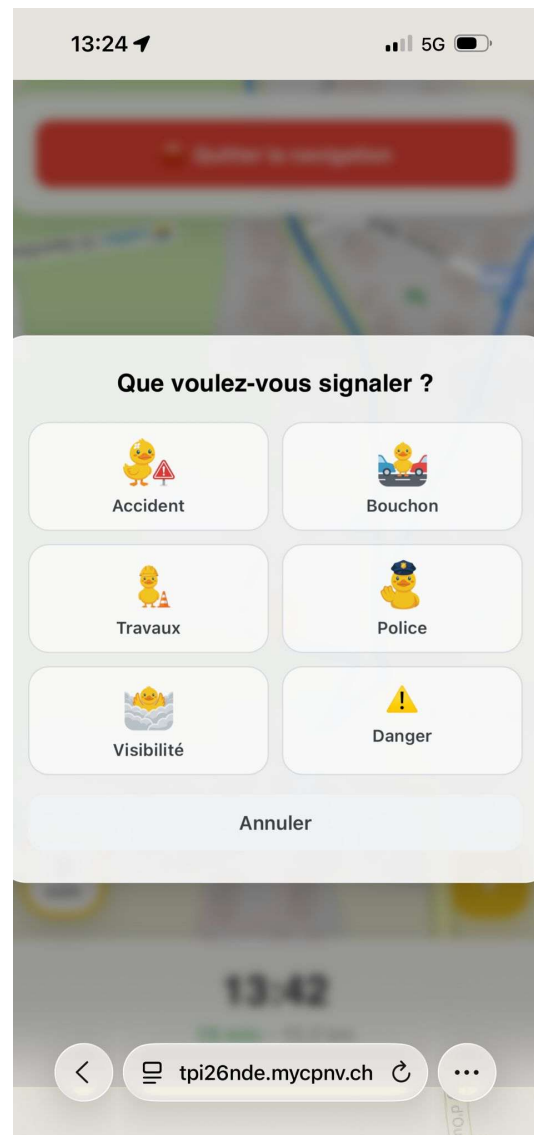


Figure 17 Siganlisation

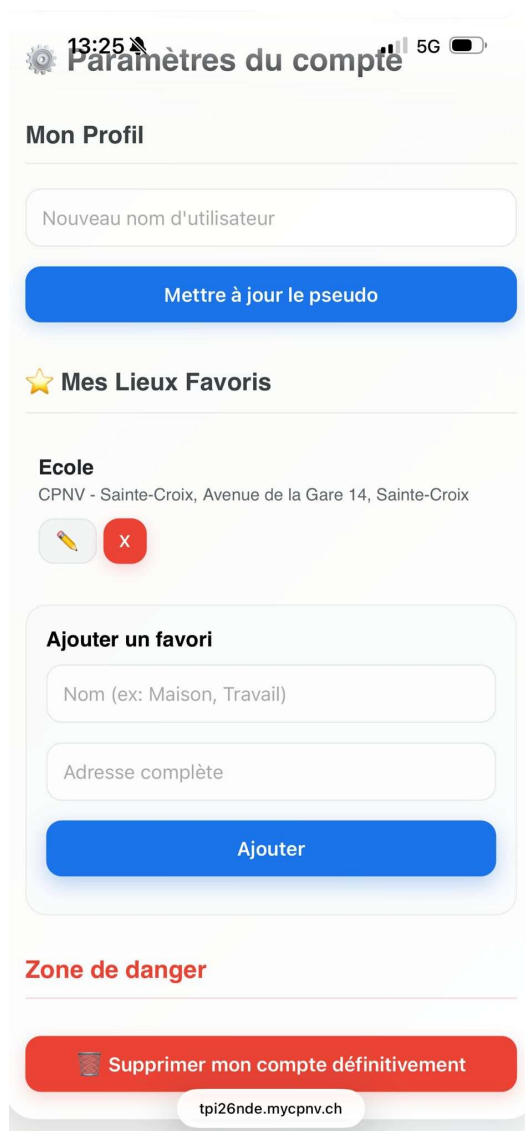


Figure 19 Settings

3.2 Description des tests effectués

La stratégie de test a reposé sur des tests fonctionnels réalisés manuellement tout au long du développement. Les conditions exactes de ces tests sont : exécution sur un navigateur avec la géolocalisation activée, en communiquant avec l'API sur Render et la base de données sur Swisscenter.

Numéro	Description	Résultats attendus	Résultats réels	Réussite / Échec	Commentaires
1.	Tests de la Base de données et Infrastructure				

Numéro	Description	Résultats attendus	Résultats réels	Réussite / Échec	Commentaires
1.1.1	Exécution du script de création de la base de données.	La base de données et les tables sont créées sans erreur.	La base est créée correctement sur Swisscenter.	Réussite	Correspond au test "Create DB"
1.1.2	Lancement de l'application et test de connexion à la base de données.	L'application communique correctement avec la base de données.	La connexion s'établit sans erreur au démarrage.	Réussite	Correspond au test "Link with DB"
2.	Tests d'authentification et d'interface				
2.1.1	Lancement de l'application pour s'inscrire ou se connecter.	L'interface de connexion et d'inscription apparaît à l'écran.	La fenêtre s'affiche correctement.	Réussite	Correspond au test "UI for login and register"
2.1.2	Inscription d'un nouvel utilisateur avec des données valides.	Un nouvel utilisateur est créé et inséré dans la base de données.	L'utilisateur est inséré avec succès en base.	Réussite	Correspond au test "Register"
2.1.3	Connexion avec un nom d'utilisateur et un mot de passe corrects.	L'utilisateur se connecte, accède à l'application et un token JWT est créé.	Connexion réussie, token généré et stocké.	Réussite	Correspond au test "Login (Working)"
2.1.4	Tentative de connexion avec un nom d'utilisateur incorrect.	La connexion est refusée et un message d'erreur apparaît.	Le message "Identifiants incorrects" s'affiche.	Réussite	Correspond au test "Login (Wrong user)"
2.1.5	Tentative de connexion avec un mot de passe incorrect.	La connexion est refusée et un message d'erreur apparaît.	Le message d'erreur s'affiche correctement.	Réussite	Correspond au test "Login (Wrong password)"

Numéro	Description	Résultats attendus	Résultats réels	Réussite / Échec	Commentaires
2.1.6	Tentative d'accès à une route protégée avec un token invalide.	L'accès est refusé et un message d'erreur est renvoyé par l'API.	L'API renvoie bien une erreur d'autorisation.	Réussite	Correspond au test "Gestion of token"
3.	Tests de la Carte et Géolocalisation				
3.1.1	Lancement de l'application après connexion.	La carte s'affiche à l'écran.	La carte (Leaflet) est visible.	Réussite	Correspond au test "Get map"
3.1.2	Récupération de la position de l'utilisateur connecté.	La position est récupérée par le navigateur et la carte se centre dessus.	Position récupérée et carte centrée.	Réussite	Correspond au test "Get user position"
3.1.3	Affichage et actualisation de l'utilisateur sur la carte.	Un marqueur ("canard") est créé à la position de l'utilisateur et le suit lors de ses déplacements.	Le marqueur se place et s'actualise correctement.	Réussite	Correspond au test "User is shown and actualisation"
3.1.4	Rotation de la carte en fonction de l'orientation de l'appareil.	La carte ou le marqueur tourne dans la direction vers laquelle l'utilisateur se dirige.	La rotation s'effectue correctement selon le cap (heading).	Réussite	Correspond au test "Rotation of map"
4.	Tests de Navigation et Itinéraires				
4.1.1	Calcul d'un itinéraire valide entre un point de départ et une destination.	Une ligne bleue apparaît sur la carte pour montrer le chemin à suivre.	Le tracé s'affiche correctement.	Réussite	Correspond au test "Create navigation road"

Numéro	Description	Résultats attendus	Résultats réels	Réussite / Échec	Commentaires
4.1.2	Autocomplétion et suggestion de lieux lors de la saisie d'une adresse.	Une liste de suggestions d'adresses apparaît en dessous du champ de saisie.	Les suggestions s'affichent correctement.	Réussite	Correspond au test "Suggestion of location"
4.1.3	Conversion d'un nom de ville/adresse en coordonnées (Géocodage).	Le nom saisi est converti avec succès en coordonnées GPS.	La conversion est effectuée avec succès.	Réussite	Correspond au test "Geocoding an address (Working)"
4.1.4	Saisie d'une adresse inexistante lors de la recherche.	Un message d'erreur s'affiche indiquant que l'adresse est introuvable.	Le message d'erreur s'affiche bien.	Réussite	Correspond au test "Geocoding an address (Wrong entry)"
4.1.5	Saisie d'une destination inatteignable par la route.	Un message d'erreur apparaît indiquant que l'itinéraire est impossible à calculer.	L'erreur d'impossibilité de route s'affiche.	Réussite	Correspond au test "Geocoding an address (Can't reach destination)"
4.1.6	Ajout d'une étape intermédiaire lors de la recherche d'itinéraire.	Le trajet se met à jour pour passer par le nouveau point de passage ajouté.	Le trajet est recalculé et passe par l'étape.	Réussite	Test complémentaire
4.1.7	Lancement du mode navigation vers la destination.	L'interface de la carte s'adapte, zoome et suit activement la position de l'utilisateur.	La carte passe en mode conduite.	Réussite	Correspond au test "Activation of navigation"
4.1.8	Affichage des données ETA lors de la	Le temps restant et la distance restante sont	Le panneau ETA s'affiche et se met à jour.	Réussite	Correspond au test "Print datas infos"

Numéro	Description	Résultats attendus	Résultats réels	Réussite / Échec	Commentaires
	navigation (conduite).	affichés et s'actualisent en temps réel.			
4.1.9	Maintien de l'écran allumé pendant la navigation.	L'écran du téléphone ne se verrouille pas lorsque l'application est en mode navigation.	L'API WakeLock maintient l'écran actif.	Réussite	Correspond au test "Screen stay awake"
4.1.10	Affichage de la vitesse actuelle lors d'un déplacement.	La vitesse de l'utilisateur s'affiche sur l'interface lorsqu'il conduit.	La vitesse s'affiche et s'actualise correctement.	Réussite	Correspond au test "Get current speed"
5.	Tests de signalement d'incidents				
5.1.1	Création d'un incident depuis l'interface lors de la conduite.	L'incident est sauvegardé dans la base de données après clic sur le bouton.	L'incident est bien enregistré.	Réussite	Correspond au test "Register incident"
5.1.2	Affichage d'un incident existant sur la carte.	Lorsqu'un incident est à proximité ou sur la carte, il est visible pour l'utilisateur.	L'icône de l'incident apparaît sur la carte.	Réussite	Correspond au test "Get incident"
5.1.3	Vérification de la proximité d'un incident actif lors d'un déplacement.	Une modale de vérification s'affiche demandant si l'incident est toujours présent à l'approche (<50m).	La modale de vérification s'affiche correctement.	Réussite	Test complémentaire
6.	Tests Profil et Paramètres				

Numéro	Description	Résultats attendus	Résultats réels	Réussite / Échec	Commentaires
6.1.1	Sauvegarde d'un lieu important (favori) avec un label.	L'adresse et son nom sont sauvegardés dans la base de données.	Le favori est enregistré avec succès.	Réussite	Correspond au test "Save important place"
6.1.2	Récupération et utilisation d'un lieu important (favori).	En tapant le nom du lieu sauvegardé, il est reconnu et utilisé pour l'itinéraire.	Le favori est proposé lors de l'autocomplétion.	Réussite	Correspond au test "Get important place"
6.1.3	Changement de la langue depuis la page de connexion (Index).	Les textes de l'interface d'accueil se traduisent instantanément.	Traduction dynamique fonctionnelle.	Réussite	Correspond au test "gestion of langage (Working)"
6.1.4	Changement de la langue depuis les paramètres (Settings).	Les textes se traduisent et la préférence est sauvegardée dans la base de données de l'utilisateur.	Traduction appliquée et enregistrée en BDD.	Réussite	Correspond au test "gestion of langage (Working on settings)"
6.1.5	Suppression définitive du compte depuis les paramètres.	En cliquant sur la suppression, le compte utilisateur est effacé de la DB.	Le compte est correctement supprimé.	Réussite	Correspond au test "Deletion of account"

3.3 Erreurs restantes

Malgré le bon fonctionnement général de l'application, quelques défauts mineurs ou limites techniques persistent :

- Dépendance à une API de géocodage gratuite : Le géocodage et l'autocomplétion dépendent de l'API publique photon.komoot.io.

Conséquences : En cas de saisie très rapide dans les champs, l'API peut temporairement bloquer les requêtes (limite de *rate-limiting*), empêchant l'affichage des suggestions d'adresses.

Actions envisagées : Implémenter une mémoire cache côté serveur (sur Render) ou un système de temporisation (*debounce*) plus long côté client en JavaScript pour espacer artificiellement les requêtes.

- Décalage visuel du marqueur principal (Canard) : La caméra utilise un correctif manuel (`map.panBy([0, -150])`) afin de décaler l'icône sous l'interface flottante (glassmorphism) de la page d'accueil.

Conséquences : Lors d'un changement d'orientation de l'appareil ou sur certains écrans de très petite taille, le centrage de l'utilisateur peut parfois paraître imprécis ou l'icône peut être légèrement masquée par les contrôles.

Actions envisagées : Utiliser les options natives de Leaflet d'offset (*padding*) lors de l'appel à `map.fitBounds` ou `map.setView` pour garantir une gestion responsive et optique parfaite, indépendante de la taille de l'écran.

3.4 Liste des documents fournis

Les documents suivants sont fournis pour garantir la bonne compréhension, le déploiement et la maintenance du produit :

- Le rapport de projet final (ce document).
- Le journal de travail détaillé.
- Le manuel d'Installation (en annexe) : contenant la procédure de mise en place de la base de données sur Swisscenter et le déploiement sur Render.
- Le manuel d'Utilisation avec des exemples graphiques (en annexe) : explicitant le fonctionnement de la carte et du système de signalement collaboratif.
- Le code source complet du logiciel (Frontend et Backend) livré sous la forme d'un lien vers le dépôt distant GitHub ou d'une archive contenant l'application complète et le script SQL de création de la base de données.

4 Conclusions

Le projet est arrivé à son terme et a permis de développer un produit fonctionnel démontrant les concepts clés demandés, tels que la cartographie, la localisation et le signalement collaboratif.

4.1 Objectifs atteints / non-atteints

La grande majorité des fonctionnalités exigées par le cahier des charges ont été implémentées avec succès :

- **Affichage de la carte et géolocalisation** : Atteint. La carte s'affiche correctement via le service de tuiles OpenStreetMap et l'application utilise l'API du navigateur pour la localisation de l'utilisateur.

- **Calcul d'itinéraire** : Atteint. La recherche et le calcul d'itinéraire simple entre un point de départ et une destination fonctionnent grâce à l'intégration de *Leaflet Routing Machine*.
- **Signalement et visualisation d'incidents** : Atteint. Les utilisateurs peuvent signaler divers incidents sur la route (accidents, bouchons, présence policière, etc.) qui s'affichent sous forme d'icônes personnalisées sur la carte.
- **Interface web responsive** : Atteint. L'interface s'adapte parfaitement aux smartphones, avec une conception ergonomique moderne.
- **Mises à jour en temps réel via WebSockets** : Non-atteint / Partiellement atteint. Le cahier des charges demandait l'utilisation de WebSockets (Socket.io) pour la synchronisation. Dans la version actuelle, cela a été remplacé par un système de rafraîchissement périodique (polling via `setInterval`) qui interroge l'API toutes les 15 secondes pour récupérer les nouveaux incidents.

4.2 Points positifs / négatifs

Points positifs :

Le résultat final est une application fluide et agréable à utiliser. L'ergonomie a été particulièrement soignée en utilisant un design moderne (effet *glassmorphism* et menus adaptés aux mobiles). De plus, j'ai pu implémenter des fonctionnalités supplémentaires qui enrichissent l'expérience utilisateur, telles que la gestion d'une liste de lieux favoris, le support multilingue (français, anglais, allemand), et l'affichage dynamique de la vitesse de l'utilisateur couplée à l'interrogation de la limite de vitesse locale via l'API Overpass. La structure du code backend (Node.js/Express) et la base de données sont claires et modulaires.

Points négatifs :

Le principal point négatif réside dans l'absence de l'intégration de Socket.io. Le choix d'utiliser du *polling* pour actualiser les incidents fonctionne bien pour un prototype, mais ce système générerait une charge serveur trop importante si l'application devait accueillir un grand nombre d'utilisateurs simultanés. De plus, le système de recalcul d'itinéraire lorsque l'utilisateur s'éloigne du trajet prévu est fonctionnel mais pourrait être optimisé pour être plus rapide et moins sensible aux petites imprécisions du signal GPS.

4.3 Difficultés particulières

Durant la phase d'implémentation, j'ai été confronté à plusieurs défis techniques notables :

- **L'autocomplétion hybride des adresses** : Combiner de manière fluide les résultats provenant d'API de géocodage externes (Photon / Nominatim) avec les adresses favorites stockées dans la base de données de l'utilisateur a nécessité une logique JavaScript complexe pour éviter les doublons et gérer les requêtes asynchrones sans figer l'interface.
- **La manipulation de la carte en mode navigation** : Contrairement à un affichage statique, faire pivoter la carte entière et l'icône de l'utilisateur de

manière fluide en fonction de la direction de déplacement (heading du GPS) a demandé de nombreux ajustements CSS et JavaScript.

- **Gestion de l'affichage des incidents (Clustering)** : Pour éviter que la carte ne devienne illisible lorsque de nombreux utilisateurs signalent le même incident au même endroit, j'ai dû concevoir un algorithme client permettant de regrouper visuellement les incidents de même type situés dans un rayon de 50 mètres.
-

4.4 Suites possibles pour le projet (évolutions & améliorations)

L'application constitue une excellente base qui pourrait être largement étendue si le projet devait se poursuivre. Voici les principales améliorations envisageables :

1. **Implémentation des WebSockets** : Remplacer le système d'actualisation actuel par Socket.io pour permettre une véritable communication bidirectionnelle en temps réel, garantissant l'apparition instantanée d'un incident chez tous les utilisateurs à proximité.
2. **Routage dynamique basé sur le trafic communautaire** : Interconnecter les signalements d'incidents (comme une route totalement bloquée) avec l'algorithme de calcul d'itinéraire afin que Leaflet propose automatiquement un chemin alternatif évitant la zone signalée.
3. **Guidage vocal (Turn-by-turn)** : Intégrer l'API de synthèse vocale des navigateurs (Web Speech API) pour dicter les instructions de direction à l'utilisateur, lui permettant de se concentrer sur la route. Ou des alertes lors d'excès de vitesse ou concernant les incidents sur la route suivie.
4. **Système de réputation des utilisateurs (Gamification)** : Mettre en place un système de points et de niveaux pour les utilisateurs dont les signalements sont fréquemment confirmés par d'autres, permettant de filtrer plus efficacement les faux signalements.

5 Annexes

5.1 Résumé du rapport du TPI / version succincte de la documentation

Le projet **DuckyRoad** a été développé dans le cadre de mon Travail Pratique Individuel (TPI). L'objectif principal était de concevoir et développer une application web de navigation et de signalement collaboratif en temps réel, inspirée de systèmes existants comme Waze, mais axée sur un produit minimum viable (MVP).

L'application a été réalisée avec une architecture full-stack JavaScript : un backend en **Node.js avec le framework Express.js** et une base de données **MySQL**, communiquant avec un frontend en HTML/CSS/JavaScript vanilla. La cartographie et le calcul d'itinéraires reposent sur les bibliothèques **Leaflet** et **Leaflet Routing Machine**, couplées aux API de géocodage ouvertes (Nominatim, Photon) et à Overpass API pour les limitations de vitesse.

L'application propose une gestion complète et sécurisée des utilisateurs (inscription, connexion par JWT), la personnalisation du profil (langue, favoris), ainsi qu'un système de navigation dynamique centré sur la position GPS de l'utilisateur. La fonctionnalité clé réside dans la possibilité de signaler des incidents (accidents, bouchons, police, etc.) visibles par les autres utilisateurs et la vérification collaborative de leur présence. L'ensemble des objectifs fixés dans le cahier des charges ont été atteints. L'application offre une interface ergonomique, réactive et adaptée aux supports mobiles. Le projet final est fonctionnel et entièrement déployé en production : **le frontend (pages HTML/CSS/JS) ainsi que la base de données MySQL sont hébergés sur Swisscenter, tandis que le backend (API Node.js) est hébergé et exécuté sur Render.**

5.2 Sources – Bibliographie

Documentation technique et frameworks :

- **Node.js & Express.js** (2026) : Documentation officielle pour la création de l'API REST. (URL : <https://nodejs.org/fr/docs/> / <https://expressjs.com/>)
- **Leaflet & Leaflet Routing Machine** (2026) : Documentation officielle pour l'affichage de la carte et la gestion du routage. (URL : <https://leafletjs.com/reference.html> / <https://www.liedman.net/leaflet-routing-machine/>)
- **Nominatim & Photon (Komoot)** : API de géocodage pour la recherche d'adresses et l'autocomplétion. (URL : <https://nominatim.org/> / <https://photon.komoot.io/>)
- **Overpass API** : Récupération des données OpenStreetMap pour les limitations de vitesse. (URL : <https://overpass-api.de/>)

- **MySQL2, Bcrypt, JsonWebToken** : Documentations des paquets NPM utilisés pour la base de données et la sécurité. (URL : <https://www.npmjs.com/>)

Outils d'assistance, hébergement et ressources graphiques :

- **MDN Web Docs** : Référence pour le HTML, CSS et JavaScript vanilla. (URL : <https://developer.mozilla.org/fr/>)
- **Swisscenter & Render** : Plateformes utilisées pour l'hébergement de la base de données, du frontend et du backend.
- **StackOverflow** : Aide à la résolution de bugs techniques spécifiques (géolocalisation, promesses asynchrones).
- **Flaticon / Freepik / Créations personnelles** : Base pour les icônes des incidents (canards personnalisés) et l'UI.

Aides humaines :

- **Yann Saison (Chef de projet)** : Conseils techniques, validation des étapes et retours durant le projet.
- **Kendra Pfändler (testeuse)** : Aide au test de l'application et retours utilisateur.

5.3 Journal de travail

Date	Durée	Activité	Remarques
02.02.2026	03h15	Analyse du CDC, planification et conception de la base de données	Création du MCD et MLD validés par le CdP.
03.02.2026	05h00	Initialisation du projet (Node.js, Express) et repo Git	Configuration de package.json, installation des dépendances (cors, dotenv, mysql2).
04.02.2026	01h35	Création de la DB distante et architecture backend	Script SQL de base. Connexion via le pool MySQL2 réussie.
09.02.2026	03h15	Début de l'implémentation de l'authentification	Structure des requêtes de connexion et création de l'interface index.html.

Date	Durée	Activité	Remarques
10.02.2026	05h00	Finalisation de l'authentification (API + Frontend)	Mise en place de Bcrypt pour le hash et JWT pour les tokens sécurisés.
11.02.2026	01h35	Début de l'intégration Leaflet	Préparation du conteneur de la carte et chargement des tuiles.
16.02.2026	03h15	Géolocalisation de l'utilisateur	La carte s'affiche. Le suivi GPS avec watchPosition fonctionne mais le curseur tremble un peu.
17.02.2026	05h00	Calcul d'itinéraires et géocodage (Nominatim/Photon)	Difficultés avec certaines adresses mal reconnues. Ajout de Photon pour améliorer l'autocomplétion.
18.02.2026	01h35	Finalisation des itinéraires	Tests d'itinéraires avec points de départ et d'arrivée dynamiques.
23.02.2026	03h15	Mode Navigation et suivi utilisateur	Rotations de la carte basées sur le heading du GPS. Interface de guidage masquant les menus.
24.02.2026	05h00	Gestion des incidents (Backend et DB)	Création des routes API (/incidents) pour stocker et récupérer les signalements.
25.02.2026	01h35	Récupération des incidents (Frontend)	Connexion du frontend à l'API pour charger les incidents environnants.
02.03.2026	03h15	Affichage des incidents sur la carte	Clustering manuel des incidents proches développé pour éviter la superposition des marqueurs.

Date	Durée	Activité	Remarques
03.03.2026	05h00	Système de vérification des incidents (Prompt utilisateur)	Ajout d'une pop-up si le GPS détecte un incident à moins de 50m.
04.03.2026	01h35	Début des paramètres utilisateur	Création de l'interface de la page settings.html.
09.03.2026	03h15	Gestion des favoris	Les favoris utilisateurs sont enregistrés et pris en compte dans l'autocomplétion.
10.03.2026	05h00	Ajout des limitations de vitesse (Overpass API)	La limitation de vitesse s'affiche et clignote en rouge en cas d'excès.
11.03.2026	01h35	Préparation du multilinguisme	Structuration des fichiers JSON pour la gestion des langues.
16.03.2026	03h15	Multilinguisme (Français, Anglais, Allemand)	Fichier translations.js implémenté. Traduction complète de l'interface.
17.03.2026	05h00	UI/UX (Glassmorphism) et correction de bugs mineurs	Amélioration du design global (arrondis, transparence). Correction du bug du canard sous le menu en navigation.
18.03.2026	01h35	Optimisation responsive	Ajustements CSS pour garantir un bon affichage sur les formats mobiles.
23.03.2026	03h15	Tests finaux de l'application	Tests complets d'itinéraires et de création d'incidents en condition réelle.

Date	Durée	Activité	Remarques
24.03.2026	05h00	Nettoyage du code et préparation documentaire	Refactoring du code, suppression des console.log et commentaires des fonctions complexes.
25.03.2026	01h35	Structuration du rapport	Mise en page du document et insertion des captures d'écran.
27.03.2026	05h00	Rédaction de la documentation finale (Passe-droit)	Rédaction des manuels d'installation, d'utilisation et bilan personnel.
28.03.2026	03h00	Relecture et dépôt final (Passe-droit)	Préparation des annexes, du README, génération de l'archive ZIP et relecture du rapport de projet.

5.4 Manuel d'Installation

- **Prérequis (pour une installation locale)**
- **Node.js** (version 18 ou supérieure recommandée) et **npm** installés sur la machine.
- Un serveur **MySQL** (local via XAMPP/WAMP ou distant).
- Git installé.
- **Étapes d'installation locale**
- 1. **Cloner le dépôt** : Ouvrir un terminal et exécuter la commande suivante : `git clone https://github.com/hev0s/PreTpi-Navigation.git` `cd PreTpi-Navigation`
- 2. **Installer les dépendances** : Exécuter la commande pour installer les paquets nécessaires au backend : `npm install`
- 3. **Configurer la base de données** : Importer le fichier contenant la structure de la base de données (`database.sql` fourni dans l'archive) dans votre serveur MySQL.

4. **Configurer les variables d'environnement** : Créer un fichier nommé `.env` à la racine du projet et y renseigner les informations de connexion :

Extrait de code

```
DB_HOST=votre_hote_mysql
DB_USER=votre_utilisateur
DB_PASSWORD=votre_mot_de_passe
DB_NAME=nom_de_la_base
JWT_SECRET=votre_cle_secrete_jwt
PORT=3000
```

Note : En production, le frontend communique avec l'API hébergée sur Render, et la base de données est gérée par Swisscenter.

5. **Démarrer le serveur** : Lancer l'application via la commande : `npm start`
L'application sera accessible localement sur le port configuré.

5.5 Manuel d'Utilisation

1. **Accéder à l'application** : Ouvrez un navigateur Web (Chrome, Safari ou Firefox recommandés) et accédez à l'URL de production hébergée sur Swisscenter (<https://tpi26nde.mycpnv.ch>) L'utilisation sur un smartphone avec le GPS activé est indispensable pour profiter pleinement des fonctionnalités.
2. **Créer un compte ou se connecter** : Sur la page d'accueil, créez un compte. Une fois connecté, acceptez la demande d'autorisation de géolocalisation demandée par votre navigateur.
3. **Calculer un itinéraire** : Dans le panneau supérieur, saisissez votre destination. Si vous souhaitez ajouter des étapes, cliquez sur + Ajouter une étape. Cliquez ensuite sur **"Rechercher l'itinéraire"**.
4. **Lancer la navigation** : Une fois le tracé bleu généré sur la carte, cliquez sur  **Démarrer la navigation**". L'interface bascule en mode conduite, affichant votre vitesse en temps réel et votre heure d'arrivée estimée.
5. **Signaler un incident** : En cours de navigation, utilisez le bouton jaune "+" en bas à droite de l'écran pour ouvrir le menu de signalement et avertir la communauté d'un danger (Bouchon, Accident, Contrôle de police, etc.). Les données sont envoyées instantanément au serveur Render pour être affichées chez les autres utilisateurs.
6. **Gérer ses paramètres** : Cliquez sur l'icône d'engrenage (⚙️) pour accéder à vos paramètres. Vous pouvez y changer la langue de l'interface, mettre à jour votre profil ou enregistrer des adresses favorites (ex: "Maison", "Travail") pour accélérer vos futures recherches.

5.6 Archives du projet

L'archive ZIP remise à la fin du projet comprend les éléments suivants :

- PreTpi-Navigation / : Dossier contenant le code source complet de l'application (Backend Node.js pour Render et Frontend HTML/CSS/JS pour Swisscenter).
- database.sql : Script de création de la base de données MySQL.
- TPI_Delafontaine_Niels_Rapport.pdf : Le présent rapport de projet au format PDF.
- TPI_Delafontaine_Niels_Journal.pdf : Le journal de travail détaillé.
- Annexes/ : Dossier contenant les documents annexes (Maquettes de l'interface, Modèles de base de données MCD/MLD, images des icônes de navigation).

5.7 Liens vers les sites du projet :

Voilà les liens vers les différents sites externes :

- GitHub : <https://github.com/hev0s/PreTpi-Navigation/tree/main>
- IceScrum : <https://icescrum.cpnv.ch/p/NAVIGATOR/#/project>
- Swisscenter : <https://apanel.swisscenter.com/>
- Render : <https://dashboard.render.com/>
- Le site : <https://tpi26nde.mycpnv.ch/>

5.8 Table des illustrations

Figure 1 Cahier des charges	5
Figure 2 Cahier des charges	6
Figure 3 Cahier des charges	7
Figure 4 FlowChart	13
Figure 5 MCD	14
Figure 6 MLD	18
Figure 7 Maquette Connexion	19
Figure 8 Maquette Inscription	20
Figure 9 Maquette Home	21
Figure 10 Maquette Navigation	22
Figure 11 Maquette Signalisation	23
Figure 12 Maquette Settings	24
Figure 13 Login	31
Figure 14 Inscription	31
Figure 15 Start Navigation	32
Figure 16 Home	32
Figure 17 Siganlisation	33
Figure 18 Navigation	33

Figure 19 Settings	34
--------------------------	----